

STAT 204 HW 1

Setup

```
#Loading package  
library(palmerpenguins)
```

Attaching package: 'palmerpenguins'

The following objects are masked from 'package:datasets':

penguins, penguins_raw

Exploring the Dataset

```
#get number of rows and columns  
dim(penguins_raw)
```

```
[1] 344 17
```

```
# 344 rows, 8 columns  
#get variable types  
str(penguins)
```

```
tibble [344 x 8] (S3: tbl_df/tbl/data.frame)
 $ species      : Factor w/ 3 levels "Adelie","Chinstrap",...: 1 1 1 1 1 1 1 1 1 1 ...
 $ island       : Factor w/ 3 levels "Biscoe","Dream",...: 3 3 3 3 3 3 3 3 3 3 ...
 $ bill_length_mm : num [1:344] 39.1 39.5 40.3 NA 36.7 39.3 38.9 39.2 34.1 42 ...
 $ bill_depth_mm  : num [1:344] 18.7 17.4 18 NA 19.3 20.6 17.8 19.6 18.1 20.2 ...
 $ flipper_length_mm: int [1:344] 181 186 195 NA 193 190 181 195 193 190 ...
 $ body_mass_g    : int [1:344] 3750 3800 3250 NA 3450 3650 3625 4675 3475 4250 ...
 $ sex           : Factor w/ 2 levels "female","male": 2 1 1 NA 1 2 1 2 NA NA ...
 $ year          : int [1:344] 2007 2007 2007 2007 2007 2007 2007 2007 2007 2007 ...
```

```
# A mix of variable types: 3 factors, 2 numeric, 3 integer
```

Part 1: Vectors and Summary Statistics

```
# 1.1 Extract and analyze bill lengths
#Extract bill_length_mm as a vector
bill_lengths <- penguins$bill_length_mm
#Calculate summary statistics of bill lengths (mean, median, standard dev, min, max, respect.
mean(bill_lengths, na.rm = TRUE)
```

```
[1] 43.92193
```

```
median(bill_lengths, na.rm = TRUE)
```

```
[1] 44.45
```

```
sd(bill_lengths, na.rm = TRUE)
```

```
[1] 5.459584
```

```
min(bill_lengths, na.rm = TRUE)
```

```
[1] 32.1
```

```
max(bill_lengths, na.rm = TRUE)
```

```
[1] 59.6
```

```
# 1.2 Logical subsetting
# Create logical vector: TRUE when bill length > 45mm
long_bills <- bill_lengths[bill_lengths >= 45]
# How many penguins have long bills?
length(long_bills)
```

```
[1] 168
```

Part 2: Subsetting Data

```
# 2.1 Island Analysis
# Extract bill lengths for Dream Island Penguins only
dream_bill_lengths <- bill_lengths[penguins$island == "Dream"]
length(dream_bill_lengths)
```

```
[1] 124
```

```
# Calculate the mean for this subset
mean(dream_bill_lengths, na.rm = TRUE)
```

```
[1] 44.16774
```

```
# 2.2 Species Counting
# How many Adelie penguins are in the dataset?
length(penguins$species[penguins$species == "Adelie"])
```

```
[1] 152
```

```
# 2.3 Find the heaviest penguin
# Find body mass of heaviest penguin
body_mass <- penguins$body_mass_g
which.max(body_mass)
```

```
[1] 170
```

```
print(penguins[170, ])
```

```
# A tibble: 1 x 8
  species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
  <fct>   <fct>         <dbl>         <dbl>             <int>       <int>
1 Gentoo  Biscoe           49.2           15.2             221        6300
# i 2 more variables: sex <fct>, year <int>
```

```
# This will generate Row 170, which is the individual with the largest body mass, and provide
```

Part 3: Matrices and apply()

```
# 3.1 Create measurement matrix
# Remove rows with missing values
penguins_complete <- penguins[complete.cases(penguins), ]
# Create matrix with: bill_length_mm, bill_depth_mm, flipper_length_mm, body_mass_g
bill_lengths <- penguins_complete$bill_length_mm
bill_depths <- penguins_complete$bill_depth_mm
flipper_lengths <- penguins_complete$flipper_length_mm
body_mass <- penguins_complete$body_mass_g
penguins_measurements <- cbind(penguins_complete$bill_length_mm, penguins_complete$bill_depth_mm,
penguins_complete$flipper_length_mm, penguins_complete$body_mass_g)
penguins_measurements
```

```
      [,1] [,2] [,3] [,4]
[1,] 39.1 18.7 181 3750
[2,] 39.5 17.4 186 3800
[3,] 40.3 18.0 195 3250
[4,] 36.7 19.3 193 3450
[5,] 39.3 20.6 190 3650
[6,] 38.9 17.8 181 3625
[7,] 39.2 19.6 195 4675
[8,] 41.1 17.6 182 3200
[9,] 38.6 21.2 191 3800
[10,] 34.6 21.1 198 4400
[11,] 36.6 17.8 185 3700
[12,] 38.7 19.0 195 3450
[13,] 42.5 20.7 197 4500
[14,] 34.4 18.4 184 3325
```

[15,]	46.0	21.5	194	4200
[16,]	37.8	18.3	174	3400
[17,]	37.7	18.7	180	3600
[18,]	35.9	19.2	189	3800
[19,]	38.2	18.1	185	3950
[20,]	38.8	17.2	180	3800
[21,]	35.3	18.9	187	3800
[22,]	40.6	18.6	183	3550
[23,]	40.5	17.9	187	3200
[24,]	37.9	18.6	172	3150
[25,]	40.5	18.9	180	3950
[26,]	39.5	16.7	178	3250
[27,]	37.2	18.1	178	3900
[28,]	39.5	17.8	188	3300
[29,]	40.9	18.9	184	3900
[30,]	36.4	17.0	195	3325
[31,]	39.2	21.1	196	4150
[32,]	38.8	20.0	190	3950
[33,]	42.2	18.5	180	3550
[34,]	37.6	19.3	181	3300
[35,]	39.8	19.1	184	4650
[36,]	36.5	18.0	182	3150
[37,]	40.8	18.4	195	3900
[38,]	36.0	18.5	186	3100
[39,]	44.1	19.7	196	4400
[40,]	37.0	16.9	185	3000
[41,]	39.6	18.8	190	4600
[42,]	41.1	19.0	182	3425
[43,]	36.0	17.9	190	3450
[44,]	42.3	21.2	191	4150
[45,]	39.6	17.7	186	3500
[46,]	40.1	18.9	188	4300
[47,]	35.0	17.9	190	3450
[48,]	42.0	19.5	200	4050
[49,]	34.5	18.1	187	2900
[50,]	41.4	18.6	191	3700
[51,]	39.0	17.5	186	3550
[52,]	40.6	18.8	193	3800
[53,]	36.5	16.6	181	2850
[54,]	37.6	19.1	194	3750
[55,]	35.7	16.9	185	3150
[56,]	41.3	21.1	195	4400
[57,]	37.6	17.0	185	3600

[58,]	41.1	18.2	192	4050
[59,]	36.4	17.1	184	2850
[60,]	41.6	18.0	192	3950
[61,]	35.5	16.2	195	3350
[62,]	41.1	19.1	188	4100
[63,]	35.9	16.6	190	3050
[64,]	41.8	19.4	198	4450
[65,]	33.5	19.0	190	3600
[66,]	39.7	18.4	190	3900
[67,]	39.6	17.2	196	3550
[68,]	45.8	18.9	197	4150
[69,]	35.5	17.5	190	3700
[70,]	42.8	18.5	195	4250
[71,]	40.9	16.8	191	3700
[72,]	37.2	19.4	184	3900
[73,]	36.2	16.1	187	3550
[74,]	42.1	19.1	195	4000
[75,]	34.6	17.2	189	3200
[76,]	42.9	17.6	196	4700
[77,]	36.7	18.8	187	3800
[78,]	35.1	19.4	193	4200
[79,]	37.3	17.8	191	3350
[80,]	41.3	20.3	194	3550
[81,]	36.3	19.5	190	3800
[82,]	36.9	18.6	189	3500
[83,]	38.3	19.2	189	3950
[84,]	38.9	18.8	190	3600
[85,]	35.7	18.0	202	3550
[86,]	41.1	18.1	205	4300
[87,]	34.0	17.1	185	3400
[88,]	39.6	18.1	186	4450
[89,]	36.2	17.3	187	3300
[90,]	40.8	18.9	208	4300
[91,]	38.1	18.6	190	3700
[92,]	40.3	18.5	196	4350
[93,]	33.1	16.1	178	2900
[94,]	43.2	18.5	192	4100
[95,]	35.0	17.9	192	3725
[96,]	41.0	20.0	203	4725
[97,]	37.7	16.0	183	3075
[98,]	37.8	20.0	190	4250
[99,]	37.9	18.6	193	2925
[100,]	39.7	18.9	184	3550

[101,]	38.6	17.2	199	3750
[102,]	38.2	20.0	190	3900
[103,]	38.1	17.0	181	3175
[104,]	43.2	19.0	197	4775
[105,]	38.1	16.5	198	3825
[106,]	45.6	20.3	191	4600
[107,]	39.7	17.7	193	3200
[108,]	42.2	19.5	197	4275
[109,]	39.6	20.7	191	3900
[110,]	42.7	18.3	196	4075
[111,]	38.6	17.0	188	2900
[112,]	37.3	20.5	199	3775
[113,]	35.7	17.0	189	3350
[114,]	41.1	18.6	189	3325
[115,]	36.2	17.2	187	3150
[116,]	37.7	19.8	198	3500
[117,]	40.2	17.0	176	3450
[118,]	41.4	18.5	202	3875
[119,]	35.2	15.9	186	3050
[120,]	40.6	19.0	199	4000
[121,]	38.8	17.6	191	3275
[122,]	41.5	18.3	195	4300
[123,]	39.0	17.1	191	3050
[124,]	44.1	18.0	210	4000
[125,]	38.5	17.9	190	3325
[126,]	43.1	19.2	197	3500
[127,]	36.8	18.5	193	3500
[128,]	37.5	18.5	199	4475
[129,]	38.1	17.6	187	3425
[130,]	41.1	17.5	190	3900
[131,]	35.6	17.5	191	3175
[132,]	40.2	20.1	200	3975
[133,]	37.0	16.5	185	3400
[134,]	39.7	17.9	193	4250
[135,]	40.2	17.1	193	3400
[136,]	40.6	17.2	187	3475
[137,]	32.1	15.5	188	3050
[138,]	40.7	17.0	190	3725
[139,]	37.3	16.8	192	3000
[140,]	39.0	18.7	185	3650
[141,]	39.2	18.6	190	4250
[142,]	36.6	18.4	184	3475
[143,]	36.0	17.8	195	3450

[144,]	37.8	18.1	193	3750
[145,]	36.0	17.1	187	3700
[146,]	41.5	18.5	201	4000
[147,]	46.1	13.2	211	4500
[148,]	50.0	16.3	230	5700
[149,]	48.7	14.1	210	4450
[150,]	50.0	15.2	218	5700
[151,]	47.6	14.5	215	5400
[152,]	46.5	13.5	210	4550
[153,]	45.4	14.6	211	4800
[154,]	46.7	15.3	219	5200
[155,]	43.3	13.4	209	4400
[156,]	46.8	15.4	215	5150
[157,]	40.9	13.7	214	4650
[158,]	49.0	16.1	216	5550
[159,]	45.5	13.7	214	4650
[160,]	48.4	14.6	213	5850
[161,]	45.8	14.6	210	4200
[162,]	49.3	15.7	217	5850
[163,]	42.0	13.5	210	4150
[164,]	49.2	15.2	221	6300
[165,]	46.2	14.5	209	4800
[166,]	48.7	15.1	222	5350
[167,]	50.2	14.3	218	5700
[168,]	45.1	14.5	215	5000
[169,]	46.5	14.5	213	4400
[170,]	46.3	15.8	215	5050
[171,]	42.9	13.1	215	5000
[172,]	46.1	15.1	215	5100
[173,]	47.8	15.0	215	5650
[174,]	48.2	14.3	210	4600
[175,]	50.0	15.3	220	5550
[176,]	47.3	15.3	222	5250
[177,]	42.8	14.2	209	4700
[178,]	45.1	14.5	207	5050
[179,]	59.6	17.0	230	6050
[180,]	49.1	14.8	220	5150
[181,]	48.4	16.3	220	5400
[182,]	42.6	13.7	213	4950
[183,]	44.4	17.3	219	5250
[184,]	44.0	13.6	208	4350
[185,]	48.7	15.7	208	5350
[186,]	42.7	13.7	208	3950

[187,]	49.6	16.0	225	5700
[188,]	45.3	13.7	210	4300
[189,]	49.6	15.0	216	4750
[190,]	50.5	15.9	222	5550
[191,]	43.6	13.9	217	4900
[192,]	45.5	13.9	210	4200
[193,]	50.5	15.9	225	5400
[194,]	44.9	13.3	213	5100
[195,]	45.2	15.8	215	5300
[196,]	46.6	14.2	210	4850
[197,]	48.5	14.1	220	5300
[198,]	45.1	14.4	210	4400
[199,]	50.1	15.0	225	5000
[200,]	46.5	14.4	217	4900
[201,]	45.0	15.4	220	5050
[202,]	43.8	13.9	208	4300
[203,]	45.5	15.0	220	5000
[204,]	43.2	14.5	208	4450
[205,]	50.4	15.3	224	5550
[206,]	45.3	13.8	208	4200
[207,]	46.2	14.9	221	5300
[208,]	45.7	13.9	214	4400
[209,]	54.3	15.7	231	5650
[210,]	45.8	14.2	219	4700
[211,]	49.8	16.8	230	5700
[212,]	49.5	16.2	229	5800
[213,]	43.5	14.2	220	4700
[214,]	50.7	15.0	223	5550
[215,]	47.7	15.0	216	4750
[216,]	46.4	15.6	221	5000
[217,]	48.2	15.6	221	5100
[218,]	46.5	14.8	217	5200
[219,]	46.4	15.0	216	4700
[220,]	48.6	16.0	230	5800
[221,]	47.5	14.2	209	4600
[222,]	51.1	16.3	220	6000
[223,]	45.2	13.8	215	4750
[224,]	45.2	16.4	223	5950
[225,]	49.1	14.5	212	4625
[226,]	52.5	15.6	221	5450
[227,]	47.4	14.6	212	4725
[228,]	50.0	15.9	224	5350
[229,]	44.9	13.8	212	4750

[230,]	50.8	17.3	228	5600
[231,]	43.4	14.4	218	4600
[232,]	51.3	14.2	218	5300
[233,]	47.5	14.0	212	4875
[234,]	52.1	17.0	230	5550
[235,]	47.5	15.0	218	4950
[236,]	52.2	17.1	228	5400
[237,]	45.5	14.5	212	4750
[238,]	49.5	16.1	224	5650
[239,]	44.5	14.7	214	4850
[240,]	50.8	15.7	226	5200
[241,]	49.4	15.8	216	4925
[242,]	46.9	14.6	222	4875
[243,]	48.4	14.4	203	4625
[244,]	51.1	16.5	225	5250
[245,]	48.5	15.0	219	4850
[246,]	55.9	17.0	228	5600
[247,]	47.2	15.5	215	4975
[248,]	49.1	15.0	228	5500
[249,]	46.8	16.1	215	5500
[250,]	41.7	14.7	210	4700
[251,]	53.4	15.8	219	5500
[252,]	43.3	14.0	208	4575
[253,]	48.1	15.1	209	5500
[254,]	50.5	15.2	216	5000
[255,]	49.8	15.9	229	5950
[256,]	43.5	15.2	213	4650
[257,]	51.5	16.3	230	5500
[258,]	46.2	14.1	217	4375
[259,]	55.1	16.0	230	5850
[260,]	48.8	16.2	222	6000
[261,]	47.2	13.7	214	4925
[262,]	46.8	14.3	215	4850
[263,]	50.4	15.7	222	5750
[264,]	45.2	14.8	212	5200
[265,]	49.9	16.1	213	5400
[266,]	46.5	17.9	192	3500
[267,]	50.0	19.5	196	3900
[268,]	51.3	19.2	193	3650
[269,]	45.4	18.7	188	3525
[270,]	52.7	19.8	197	3725
[271,]	45.2	17.8	198	3950
[272,]	46.1	18.2	178	3250

[273,]	51.3	18.2	197	3750
[274,]	46.0	18.9	195	4150
[275,]	51.3	19.9	198	3700
[276,]	46.6	17.8	193	3800
[277,]	51.7	20.3	194	3775
[278,]	47.0	17.3	185	3700
[279,]	52.0	18.1	201	4050
[280,]	45.9	17.1	190	3575
[281,]	50.5	19.6	201	4050
[282,]	50.3	20.0	197	3300
[283,]	58.0	17.8	181	3700
[284,]	46.4	18.6	190	3450
[285,]	49.2	18.2	195	4400
[286,]	42.4	17.3	181	3600
[287,]	48.5	17.5	191	3400
[288,]	43.2	16.6	187	2900
[289,]	50.6	19.4	193	3800
[290,]	46.7	17.9	195	3300
[291,]	52.0	19.0	197	4150
[292,]	50.5	18.4	200	3400
[293,]	49.5	19.0	200	3800
[294,]	46.4	17.8	191	3700
[295,]	52.8	20.0	205	4550
[296,]	40.9	16.6	187	3200
[297,]	54.2	20.8	201	4300
[298,]	42.5	16.7	187	3350
[299,]	51.0	18.8	203	4100
[300,]	49.7	18.6	195	3600
[301,]	47.5	16.8	199	3900
[302,]	47.6	18.3	195	3850
[303,]	52.0	20.7	210	4800
[304,]	46.9	16.6	192	2700
[305,]	53.5	19.9	205	4500
[306,]	49.0	19.5	210	3950
[307,]	46.2	17.5	187	3650
[308,]	50.9	19.1	196	3550
[309,]	45.5	17.0	196	3500
[310,]	50.9	17.9	196	3675
[311,]	50.8	18.5	201	4450
[312,]	50.1	17.9	190	3400
[313,]	49.0	19.6	212	4300
[314,]	51.5	18.7	187	3250
[315,]	49.8	17.3	198	3675

```
[316,] 48.1 16.4 199 3325
[317,] 51.4 19.0 201 3950
[318,] 45.7 17.3 193 3600
[319,] 50.7 19.7 203 4050
[320,] 42.5 17.3 187 3350
[321,] 52.2 18.8 197 3450
[322,] 45.2 16.6 191 3250
[323,] 49.3 19.9 203 4050
[324,] 50.2 18.8 202 3800
[325,] 45.6 19.4 194 3525
[326,] 51.9 19.5 206 3950
[327,] 46.8 16.5 189 3650
[328,] 45.7 17.0 195 3650
[329,] 55.8 19.8 207 4000
[330,] 43.5 18.1 202 3400
[331,] 49.6 18.2 193 3775
[332,] 50.8 19.0 210 4100
[333,] 50.2 18.7 198 3775
```

```
# 3.2 Apply Functions
# Use apply() to calculate mean of each column
class(penguins_measurements)
```

```
[1] "matrix" "array"
```

```
dim(penguins_measurements)
```

```
[1] 333 4
```

```
str(penguins_measurements)
```

```
num [1:333, 1:4] 39.1 39.5 40.3 36.7 39.3 38.9 39.2 41.1 38.6 34.6 ...
```

```
apply(penguins_measurements, 2, mean, na.rm = TRUE)
```

```
[1] 43.99279 17.16486 200.96697 4207.05706
```

```
apply(penguins_measurements, 2, max, na.rm = TRUE)
```

```
[1] 59.6 21.5 231.0 6300.0
```

```
apply(penguins_measurements, 2, which.max)
```

```
[1] 179 15 209 164
```

Part 4: Custom Function

```
# 4.1 Write standardize function
# function that removes NAs, returns (x - mean) / sd
standardize <- function(x) {(x-mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE)}
#you have to put in x in parentheses so the function knows it is calculating the mean/sd of x
bill_length_standardized <- standardize(bill_lengths)
bill_length_standardized
```

```
[1] -0.894695470 -0.821551521 -0.675263622 -1.333559166 -0.858123495
[6] -0.931267445 -0.876409483 -0.528975723 -0.986125407 -1.717564900
[11] -1.351845153 -0.967839419 -0.272971901 -1.754136874 0.367037656
[16] -1.132413305 -1.150699292 -1.479847064 -1.059269356 -0.949553432
[21] -1.589562988 -0.620405660 -0.638691647 -1.114127318 -0.638691647
[26] -0.821551521 -1.242129229 -0.821551521 -0.565547698 -1.388417128
[31] -0.876409483 -0.949553432 -0.327829863 -1.168985280 -0.766693559
[36] -1.370131140 -0.583833685 -1.461561077 0.019603896 -1.278701204
[41] -0.803265533 -0.528975723 -1.461561077 -0.309543876 -0.803265533
[46] -0.711835597 -1.644420950 -0.364401838 -1.735850887 -0.474117761
[51] -0.912981457 -0.620405660 -1.370131140 -1.168985280 -1.516419039
[56] -0.492403749 -1.168985280 -0.528975723 -1.388417128 -0.437545787
[61] -1.552991014 -0.528975723 -1.479847064 -0.400973812 -1.918710760
[66] -0.784979546 -0.803265533 0.330465681 -1.552991014 -0.218113939
[71] -0.565547698 -1.242129229 -1.424989102 -0.346115850 -1.717564900
[76] -0.199827952 -1.333559166 -1.626134963 -1.223843242 -0.492403749
[81] -1.406703115 -1.296987191 -1.040983369 -0.931267445 -1.516419039
[86] -0.528975723 -1.827280824 -0.803265533 -1.424989102 -0.583833685
[91] -1.077555343 -0.675263622 -1.991854709 -0.144969990 -1.644420950
[96] -0.547261711 -1.150699292 -1.132413305 -1.114127318 -0.784979546
[101] -0.986125407 -1.059269356 -1.077555343 -0.144969990 -1.077555343
```

[106]	0.293893706	-0.784979546	-0.327829863	-0.803265533	-0.236399926
[111]	-0.986125407	-1.223843242	-1.516419039	-0.528975723	-1.424989102
[116]	-1.150699292	-0.693549609	-0.474117761	-1.607848976	-0.620405660
[121]	-0.949553432	-0.455831774	-0.912981457	0.019603896	-1.004411394
[126]	-0.163255977	-1.315273178	-1.187271267	-1.077555343	-0.528975723
[131]	-1.534705026	-0.693549609	-1.278701204	-0.784979546	-0.693549609
[136]	-0.620405660	-2.174714583	-0.602119673	-1.223843242	-0.912981457
[141]	-0.876409483	-1.351845153	-1.461561077	-1.132413305	-1.461561077
[146]	-0.455831774	0.385323643	1.098477149	0.860759313	1.098477149
[151]	0.659613453	0.458467592	0.257321732	0.495039567	-0.126684002
[156]	0.513325554	-0.565547698	0.915617275	0.275607719	0.805901351
[161]	0.330465681	0.970475237	-0.364401838	0.952189250	0.403609630
[166]	0.860759313	1.135049123	0.202463770	0.458467592	0.421895617
[171]	-0.199827952	0.385323643	0.696185427	0.769329377	1.098477149
[176]	0.604755491	-0.218113939	0.202463770	2.853931932	0.933903263
[181]	0.805901351	-0.254685914	0.074461858	0.001317909	0.860759313
[186]	-0.236399926	1.025333199	0.239035744	1.025333199	1.189907085
[191]	-0.071826040	0.275607719	1.189907085	0.165891795	0.220749757
[196]	0.476753579	0.824187339	0.202463770	1.116763136	0.458467592
[201]	0.184177782	-0.035254066	0.275607719	-0.144969990	1.171621098
[206]	0.239035744	0.403609630	0.312179694	1.884774604	0.330465681
[211]	1.061905174	1.007047212	-0.090112028	1.226479060	0.677899440
[216]	0.440181605	0.769329377	0.458467592	0.440181605	0.842473326
[221]	0.641327465	1.299623009	0.220749757	0.220749757	0.933903263
[226]	1.555626832	0.623041478	1.098477149	0.165891795	1.244765047
[231]	-0.108398015	1.336194984	0.641327465	1.482482882	0.641327465
[236]	1.500768870	0.275607719	1.007047212	0.092747846	1.244765047
[241]	0.988761225	0.531611541	0.805901351	1.299623009	0.824187339
[246]	2.177350401	0.586469503	0.933903263	0.513325554	-0.419259799
[251]	1.720200718	-0.126684002	0.751043389	1.189907085	1.061905174
[256]	-0.090112028	1.372766958	0.403609630	2.031062502	0.879045301
[261]	0.586469503	0.513325554	1.171621098	0.220749757	1.080191161
[266]	0.458467592	1.098477149	1.336194984	0.257321732	1.592198806
[271]	0.220749757	0.385323643	1.336194984	0.367037656	1.336194984
[276]	0.476753579	1.409338933	0.549897529	1.464196895	0.348751668
[281]	1.189907085	1.153335110	2.561356135	0.440181605	0.952189250
[286]	-0.291257888	0.824187339	-0.144969990	1.208193072	0.495039567
[291]	1.464196895	1.189907085	1.007047212	0.440181605	1.610484794
[296]	-0.565547698	1.866488616	-0.272971901	1.281337022	1.043619187
[301]	0.641327465	0.659613453	1.464196895	0.531611541	1.738486705
[306]	0.915617275	0.403609630	1.263051034	0.275607719	1.263051034
[311]	1.244765047	1.116763136	0.915617275	1.372766958	1.061905174
[316]	0.751043389	1.354480971	0.312179694	1.226479060	-0.272971901

```
[321] 1.500768870 0.220749757 0.970475237 1.135049123 0.293893706
[326] 1.445910908 0.513325554 0.312179694 2.159064413 -0.090112028
[331] 1.025333199 1.244765047 1.135049123
```

```
#verifying that mean = 0 and sd = 1
mean(bill_length_standardized, na.rm = TRUE)
```

```
[1] -1.081551e-15
```

```
sd(bill_length_standardized, na.rm = TRUE)
```

```
[1] 1
```

Bonus: Option B

```
# Create bill length to depth ratio
bill_length_depth_ratio <- penguins$bill_length_mm / penguins$bill_depth_mm
bill_length_depth_ratio
```

```
[1] 2.090909 2.270115 2.238889      NA 1.901554 1.907767 2.185393 2.000000
[9] 1.883978 2.079208 2.210526 2.184971 2.335227 1.820755 1.639810 2.056180
[17] 2.036842 2.053140 1.869565 2.139535 2.065574 2.016043 1.869792 2.110497
[25] 2.255814 1.867725 2.182796 2.262570 2.037634 2.142857 2.365269 2.055249
[33] 2.219101 2.164021 2.141176 1.857820 1.940000 2.281081 1.948187 2.083770
[41] 2.027778 2.217391 1.945946 2.238579 2.189349 2.106383 2.163158 1.984127
[49] 2.011173 1.995283 2.237288 2.121693 1.955307 2.153846 1.906077 2.225806
[57] 2.228571 2.159574 2.198795 1.968586 2.112426 1.957346 2.211765 2.258242
[65] 2.128655 2.311111 2.191358 2.151832 2.162651 2.154639 1.763158 2.157609
[73] 2.302326 2.423280 2.028571 2.313514 2.434524 1.917526 2.248447 2.204188
[81] 2.011628 2.437500 1.952128 1.809278 2.095506 2.034483 1.861538 1.983871
[89] 1.994792 2.069149 1.983333 2.270718 1.988304 2.187845 2.092486 2.158730
[97] 2.048387 2.178378 2.055901 2.335135 1.955307 2.050000 2.356250 1.890000
[105] 2.037634 2.100529 2.244186 1.910000 2.241176 2.273684 2.309091 2.246305
[113] 2.242938 2.164103 1.913043 2.333333 2.270588 1.819512 2.100000 2.209677
[121] 2.104651 1.904040 2.364706 2.237838 2.213836 2.136842 2.204545 2.267760
[129] 2.280702 2.450000 2.150838 2.244792 1.989189 2.027027 2.164773 2.348571
[137] 2.034286 2.000000 2.242424 2.217877 2.350877 2.360465 2.070968 2.394118
[145] 2.220238 2.085561 2.107527 1.989130 2.022472 2.088398 2.105263 2.243243
[153] 3.492424 3.067485 3.453901 3.289474 3.282759 3.444444 3.109589 3.052288
```

```

[161] 3.231343 3.038961 2.985401 3.043478 3.321168 3.315068 3.136986 3.140127
[169] 3.111111 3.236842 3.186207 3.225166 3.510490 3.110345 3.206897 2.930380
[177] 3.274809 3.052980 3.111888 3.186667 3.370629 3.267974 3.091503 3.014085
[185] 3.110345 3.505882 3.317568 2.969325 3.109489 2.566474 3.235294 3.101911
[193] 3.116788 3.100000 3.306569 3.306667 3.176101 3.136691 3.273381 3.176101
[201] 3.375940 2.860759 3.281690 3.439716 3.131944 3.340000 3.229167 2.922078
[209] 3.151079 3.033333 2.979310 3.294118 3.282609 3.100671 3.287770 3.458599
[217] 3.225352 2.964286 3.208333 3.055556 3.063380 3.380000 3.180000 2.974359
[225] 3.089744 3.141892 3.093333 3.037500 3.345070 3.134969 3.275362 2.756098
[233] 3.386207 3.365385 3.246575 3.144654 3.253623 2.936416 3.013889 3.612676
[241] 3.392857 3.064706 3.166667 3.052632 3.137931 3.074534 3.027211 3.235669
[249] 3.126582 3.212329 3.361111 3.096970 3.233333 3.288235 3.045161 3.273333
[257] 3.427536 2.906832 2.836735 3.379747 3.092857 3.185430 3.322368 3.132075
[265] 2.861842 3.159509 3.276596 3.443750 2.834395 3.012346 3.445255      NA
[273] 3.272727 3.210191 3.054054 3.099379 2.597765 2.564103 2.671875 2.427807
[281] 2.661616 2.539326 2.532967 2.818681 2.433862 2.577889 2.617978 2.546798
[289] 2.716763 2.872928 2.684211 2.576531 2.515000 3.258427 2.494624 2.703297
[297] 2.450867 2.771429 2.602410 2.608247 2.608939 2.736842 2.744565 2.605263
[305] 2.606742 2.640000 2.463855 2.605769 2.544910 2.712766 2.672043 2.827381
[313] 2.601093 2.512077 2.825301 2.688442 2.512821 2.640000 2.664921 2.676471
[321] 2.843575 2.745946 2.798883 2.500000 2.754011 2.878613 2.932927 2.705263
[329] 2.641618 2.573604 2.456647 2.776596 2.722892 2.477387 2.670213 2.350515
[337] 2.661538 2.836364 2.688235 2.818182 2.403315 2.725275 2.673684 2.684492

```

```
length(bill_length_depth_ratio)
```

```
[1] 344
```

```

#Find the Bill Length Depth ratio for each species
bld_ratio_adelie <- bill_length_depth_ratio[penguins$species == "Adelie"]
mean(bld_ratio_adelie, na.rm = TRUE)

```

```
[1] 2.119726
```

```

bld_ratio_gentoo <- bill_length_depth_ratio[penguins$species == "Gentoo"]
mean(bld_ratio_gentoo, na.rm = TRUE)

```

```
[1] 3.175592
```

```
bld_ratio_chinstrap <- bill_length_depth_ratio[penguins$species == "Chinstrap"]  
mean(bld_ratio_chinstrap, na.rm = TRUE)
```

```
[1] 2.653756
```

```
# Gentoo has the highest average bill length-to-depth ratio, which is 3.175592
```